

LISTA DE VERIFICAÇÃO PARA O INSTRUMENTO DE AVALIAÇÃO PRÁTICA

Unidade Curricular: Programação de Aplicativos (PRAP)

Estratégia de Avaliação: Situação de Aprendizagem Integrada - "Da Ideia ao Código"

Tipo de Instrumento: Lista de Verificação com Atribuição de Pesos

Turma: T DESI 2025/2 N1 - 2º Semestre

Lista de Capacidades Técnicas a serem avaliadas:

CT02: Instalar ferramentas de acordo com requisitos de hardware, software e parâmetro de configuração. (Nível Cognitivo: Aplicar)

- Justificativa Pedagógica: A autonomia na preparação e configuração do ambiente de desenvolvimento (como VS Code, interpretador Python e repositórios Git) é o primeiro passo essencial para garantir que o fluxo de trabalho profissional não sofra interrupções e permita o versionamento seguro do código-fonte.

CT03: Reconhecer especificações técnicas e paradigmas de linguagem de programação. (Nível Cognitivo: Entender/Analisar no contexto)

- Justificativa Pedagógica: A compreensão profunda dos paradigmas, especialmente a transição da Programação Estruturada para a Orientada a Objetos (POO), é a base conceitual indispensável para abstrair problemas do mundo real e arquitetar softwares modernos e escaláveis.

CT04: Aplicar linguagem de programação por meio do ambiente integrado de desenvolvimento (IDE). (Nível Cognitivo: Aplicar)

- Justificativa Pedagógica: Traduzir regras de negócio teóricas em código funcional e executável é o núcleo do desenvolvimento de software. O uso eficiente da IDE para manipular variáveis, estruturas lógicas, coleções e interfaces web maximiza a produtividade do aluno na entrega do produto final.

CT05: Integrar banco de dados por meio da linguagem de programação. (Nível Cognitivo: Aplicar)

- Justificativa Pedagógica: Um sistema corporativo sem persistência definitiva perde sua utilidade. Integrar o código-fonte (Back-End) a um banco de dados relacional é vital para garantir que as informações sejam salvas, lidas, atualizadas e protegidas contra falhas temporárias.

CT06: Aplicar métodos e técnicas de programação. (Nível Cognitivo: Aplicar/Analisar)

- Justificativa Pedagógica: A aplicação de padrões arquiteturais (como Modularização, MVC e DAO), tratamento defensivo de exceções e testes unitários é o que diferencia um script amador de um software resiliente e de fácil manutenção, exigência primária do mercado de trabalho.

Lista de Verificação - Desempenho Técnico:

| Critério de Avaliação                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | CT   | Peso |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------|
| 01- Configuração inicial do ambiente de desenvolvimento (VS Code) executada adequadamente para a linguagem Python.                                                                                                                                                                                                                                                                                                                                                                                                                         | CT02 | 0,4  |
| <ul style="list-style-type: none"><li>Sim (Atende): O aluno possui o VS Code aberto no diretório correto do projeto, com a extensão do Python instalada e o interpretador selecionado corretamente. O código base roda no terminal sem erros de caminho ou de reconhecimento de comando.</li><li>Não (Não Atende): O aluno tenta rodar o arquivo solto fora de uma pasta de projeto, o terminal acusa que o comando <code>python</code> não é reconhecido, ou há erros na IDE por falta de configuração básica do interpretador.</li></ul> |      |      |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |             |            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|------------|
| <b>02</b> - Inicialização, configuração e estruturação correta do repositório Git local e remoto (commits e push).                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <b>CT02</b> | <b>0,4</b> |
| <ul style="list-style-type: none"> <li><b>Sim (Atende):</b> É possível verificar a existência da pasta oculta <code>.git</code>, e o aluno demonstra (via comando <code>git log</code> ou interface visual) que realizou <i>commits</i> descritivos. O código está sincronizado com um repositório remoto (ex: GitHub) via <code>git push</code>.</li> <li><b>Não (Não Atende):</b> O controle de versão não foi iniciado, as alterações não foram salvas em <i>commits</i>, ou o aluno não conseguiu enviar o projeto para a nuvem.</li> </ul>                             |             |            |
| <b>03</b> - Configuração e execução apropriada do ambiente virtual e bibliotecas de terceiros (ex: Flask) para o projeto.                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>CT02</b> | <b>0,4</b> |
| <ul style="list-style-type: none"> <li><b>Sim (Atende):</b> O projeto roda isolado dentro de um ambiente virtual (ex: <code>venv</code> ativo no terminal). As bibliotecas externas, como o <code>Flask</code>, foram instaladas no ambiente virtual (preferencialmente documentadas em um <code>requirements.txt</code>).</li> <li><b>Não (Não Atende):</b> O aluno instalou as dependências globalmente no computador (sem <code>venv</code>), ou o projeto apresenta o erro <code>ModuleNotFoundError</code> ao tentar importar o Flask na execução.</li> </ul>          |             |            |
| <b>04</b> - Identificação e modelagem correta de Classes baseadas no domínio do problema (atributos e métodos coerentes).                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>CT03</b> | <b>0,5</b> |
| <ul style="list-style-type: none"> <li><b>Sim (Atende):</b> O aluno utilizou a palavra reservada <code>class</code> de forma coesa (ex: <code>class Produto;</code>), e implementou o método construtor <code>__init__</code> mapeando os atributos essenciais que fazem sentido para o projeto dele (ex: <code>self.nome</code>, <code>self.preco</code>).</li> <li><b>Não (Não Atende):</b> O código é inteiramente procedural (apenas funções e variáveis soltas), ou as classes foram criadas vazias e sem atributos que representem a regra de negócio.</li> </ul>     |             |            |
| <b>05</b> - Diferenciação correta e clara entre Classes (moldes/projetos) e Objetos (instâncias) no código desenvolvido.                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>CT03</b> | <b>0,5</b> |
| <ul style="list-style-type: none"> <li><b>Sim (Atende):</b> O aluno demonstra no código a instanciação dos objetos (ex: <code>item1 = Produto("Mouse", 50)</code>) e compreende que alterar <code>item1</code> não afeta <code>item2</code>. Os métodos de instância recebem o parâmetro <code>self</code> corretamente.</li> <li><b>Não (Não Atende):</b> O aluno confunde variáveis de classe com variáveis de instância, tenta chamar métodos diretamente da classe sem instanciar, ou não utiliza o <code>self</code>, gerando erros de execução.</li> </ul>            |             |            |
| <b>06</b> - Aplicação adequada de conceitos avançados de POO, como a Herança, na arquitetura das entidades do sistema.                                                                                                                                                                                                                                                                                                                                                                                                                                                      | <b>CT03</b> | <b>0,5</b> |
| <ul style="list-style-type: none"> <li><b>Sim (Atende):</b> O projeto possui pelo menos uma relação clara de herança (ex: <code>class ClienteVip(Cliente);</code>), utilizando a função <code>super().__init__()</code> para reaproveitar atributos da classe-mãe e adicionando comportamentos específicos.</li> <li><b>Não (Não Atende):</b> O aluno duplicou o código criando classes inteiras idênticas em vez de usar herança, ou declarou a herança mas não a utilizou na prática (não sobrescreveu métodos nem usou o <code>super</code>).</li> </ul>                 |             |            |
| <b>07</b> - Utilização correta de tipos de dados primitivos e mapeamento em Coleções (Listas, Tuplas, Dicionários).                                                                                                                                                                                                                                                                                                                                                                                                                                                         | <b>CT04</b> | <b>0,6</b> |
| <ul style="list-style-type: none"> <li><b>Sim (Atende):</b> Os dados retornados do banco ou inseridos pelo usuário são agrupados logicamente em Listas <code>[]</code>, Tuplas <code>()</code> ou Dicionários <code>{}</code> (ex: uma lista de dicionários para representar vários registros de uma tabela).</li> <li><b>Não (Não Atende):</b> Uso excessivo e desorganizado de variáveis soltas (ex: <code>nome1</code>, <code>nome2</code>, <code>nome3</code> em vez de uma lista) ou erros de tipo (ex: tentar somar um texto com um número sem conversão).</li> </ul> |             |            |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |             |            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|------------|
| <b>08</b> - Implementação lógica eficaz utilizando estruturas de decisão (if/else) e laços de repetição (for/while).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <b>CT04</b> | <b>0,6</b> |
| <ul style="list-style-type: none"> <li>• <b>Sim (Atende):</b> O fluxo do sistema é bem controlado usando <b>if/elif/else</b> para validações (ex: verificar se um campo está vazio) e <b>for</b> ou <b>while</b> para iterar sobre coleções de dados (ex: percorrer a lista de produtos para exibi-los na tela).</li> <li>• <b>Não (Não Atende):</b> Lógica falha gerando "loops infinitos", uso indevido de repetições onde não é necessário, ou incapacidade de filtrar dados condicionalmente.</li> </ul>                                                                                             |             |            |
| <b>09</b> - Criação de rotas e integração web responsiva entre Front-End e Back-End utilizando o microframework Flask.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>CT04</b> | <b>0,6</b> |
| <ul style="list-style-type: none"> <li>• <b>Sim (Atende):</b> As rotas (<b>@app.route</b>) estão criadas e respondem corretamente aos métodos HTTP (GET para exibir, POST para receber dados de formulários), renderizando as páginas em HTML.</li> <li>• <b>Não (Não Atende):</b> O servidor Flask não sobe, as rotas quebram retornando erros 404 (Not Found) ou 500 (Internal Server Error), ou os formulários não conseguem enviar dados para o Back-End.</li> </ul>                                                                                                                                 |             |            |
| <b>10</b> - Estabelecimento de conexão bem-sucedida, funcional e segura com o banco de dados local (SQLite / conexao.py).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | <b>CT05</b> | <b>0,8</b> |
| <ul style="list-style-type: none"> <li>• <b>Sim (Atende):</b> Existe um módulo/arquivo dedicado (ex: <b>conexao.py</b>) utilizando a biblioteca <b>sqlite3</b>. O sistema cria fisicamente o arquivo do banco (ex: <b>banco.db</b>) e abre a conexão sem erros.</li> <li>• <b>Não (Não Atende):</b> O sistema não consegue achar o banco, o arquivo não é gerado, ou o aluno tenta salvar dados apenas em variáveis de memória (que se perdem ao fechar o programa).</li> </ul>                                                                                                                          |             |            |
| <b>11</b> - Implementação completa e funcional das operações essenciais de CRUD (Create, Read, Update, Delete).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | <b>CT05</b> | <b>0,8</b> |
| <ul style="list-style-type: none"> <li>• <b>Sim (Atende):</b> O sistema consegue Inserir (INSERT), Ler (SELECT), Atualizar (UPDATE) e Deletar (DELETE) registros. As informações inseridas no Front-End são comprovadamente salvas na tabela física.</li> <li>• <b>Não (Não Atende):</b> Falta alguma das 4 operações essenciais, ou as instruções SQL estão escritas com erros de sintaxe e falham silenciosamente na execução.</li> </ul>                                                                                                                                                              |             |            |
| <b>12</b> - Aplicação estrita de travas de segurança cibernética na manipulação de dados (uso obrigatório de <b>WHERE</b> ).                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | <b>CT05</b> | <b>0,9</b> |
| <ul style="list-style-type: none"> <li>• <b>Sim (Atende):</b> Todas as instruções de UPDATE e DELETE possuem obrigatoriamente a cláusula <b>WHERE id = ?</b>, garantindo que apenas um registro específico seja alterado/excluído.</li> <li>• <b>Não (Não Atende):</b> Erro gravíssimo (Zero automático na questão de segurança). O código possui um UPDATE ou DELETE genérico sem <b>WHERE</b>, o que resultaria na corrupção de toda a base de dados em um cenário real.</li> </ul>                                                                                                                    |             |            |
| <b>13</b> - Estruturação e modularização do software em camadas corporativas, respeitando a separação do Padrão DAO.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <b>CT06</b> | <b>1,0</b> |
| <ul style="list-style-type: none"> <li>• <b>Sim (Atende):</b> O código SQL está 100% isolado em classes específicas (ex: <b>ProdutoDAO.py</b>). O arquivo principal do Flask (Rotas) apenas chama os métodos do DAO, não possuindo nenhuma string de banco de dados (<b>INSERT</b>, <b>SELECT</b> etc.) visível nas rotas web.</li> <li>• <b>Não (Não Atende):</b> O código é "espaguete/macarrônico". O aluno misturou as regras visuais, as lógicas de negócio e as queries de banco de dados (<b>sqlite3.connect()</b>) diretamente dentro das rotas do arquivo principal (<b>app.py</b>).</li> </ul> |             |            |
| <b>14</b> - Implementação de arquitetura defensiva (blocos <b>Try/Except</b> ) para tratamento de erros e resiliência do código.                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | <b>CT06</b> | <b>1,0</b> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |      |      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------|
| <ul style="list-style-type: none"> <li>● <b>Sim (Atende):</b> Operações críticas, como a conexão com o banco de dados ou a conversão de textos de formulários para números (<code>int()</code>, <code>float()</code>), estão protegidas por blocos <code>try/except</code>. Se o usuário digitar "ABC" no campo de preço, o sistema não "crasha", mas retorna uma mensagem amigável de erro.</li> <li>● <b>Não (Não Atende):</b> O código assume que o usuário sempre digitará a informação perfeita. Qualquer erro de digitação ou falha no banco gera a tela vermelha de erro catastrófico (Traceback) no servidor.</li> </ul> |      |      |
| 15 - Criação e execução de scripts de Testes Unitários automatizados ( <code>unittest</code> ) para validação matemática de métodos.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | CT06 | 1,0  |
| <ul style="list-style-type: none"> <li>● <b>Sim (Atende):</b> O aluno criou um arquivo de teste separado (ex: <code>testes.py</code>) usando o módulo <code>unittest</code>. O teste isola funções ou métodos do DAO, roda validações com <code>assertEqual</code> (ou similares) e o terminal reporta execução com sucesso (OK).</li> <li>● <b>Não (Não Atende):</b> Não existem testes automatizados. O aluno faz o teste exclusivamente no modo "tentativa e erro" pela tela visual do sistema, o que não garante a estabilidade matemática do código em manutenções futuras.</li> </ul>                                      |      |      |
| TOTAL                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |      | 10,0 |

**Conteúdo das aulas vinculados às Capacidades Técnicas:**

**Aula 01: Onboarding e Ambiente**

- Conteúdo Teórico/Prático: Configuração do VS Code, instalação do Python, introdução ao Git, clonagem de repositório e versionamento inicial (`git commit` / `git push`).
- Vínculo com a Lista de Verificação:
  - Capacidade Avaliada: CT02
  - Critérios Atendidos: 1.1 (Configuração do ambiente e IDE) e 1.2 (Versionamento Git).

**Aula 04 e Aula 09: Fundamentos, Paradigmas e Classes**

- Conteúdo Teórico/Prático: Transição da Programação Estruturada para a Orientada a Objetos. Criação do método construtor `__init__`, compreensão de Herança e moldagem de atributos/comportamentos.
- Vínculo com a Lista de Verificação:
  - Capacidade Avaliada: CT03
  - Critérios Atendidos: 2.1 (Modelagem de Classes), 2.2 (Diferenciação de Classes/Objetos) e 2.3 (Aplicação de Herança).

**Aula 05 e Aula 06: Coleções e Fluxo de Execução**

- Conteúdo Teórico/Prático: Agrupamento de dados em Listas, Tuplas e Dicionários. Engenharia de decisões e controle de automação através de laços `for` e `while`, com desvios lógicos `if/elif/else`.
- Vínculo com a Lista de Verificação:
  - Capacidade Avaliada: CT04
  - Critérios Atendidos: 3.1 (Utilização de Coleções) e 3.2 (Estruturas de decisão e repetição).

**Aula 19 e Aula 21: Framework Web (Flask) e Formulários**

- Conteúdo Teórico/Prático: Roteamento de endpoints utilizando Flask (`@app.route`). Transição de dados do navegador via métodos HTTP (POST/GET) e interatividade do sistema web.
- Vínculo com a Lista de Verificação:
  - Capacidade Avaliada: CT04 e CT02
  - Critérios Atendidos: 3.3 (Integração Web/Flask) e 1.3 (Bibliotecas de terceiros).

## **Aula 12 e Aula 13: Operações de Banco de Dados (CRUD)**

- Conteúdo Teórico/Prático: Instruções transacionais completas (`INSERT`, `SELECT`, `UPDATE`, `DELETE`). Foco na segurança da informação e prevenção de corrupção de base com o uso da cláusula restritiva `WHERE`.
- Vínculo com a Lista de Verificação:
  - Capacidade Avaliada: CT05
  - Critérios Atendidos: 4.2 (Operações de CRUD completas) e 4.3 (Travas de segurança e `WHERE`).

## **Aula 14 e Aula 15: ORM e Arquitetura DAO**

- Conteúdo Teórico/Prático: Criação do módulo de conectividade robusto (`conexao.py`). Isolamento dos comandos de SQL do código principal da interface utilizando o Padrão de Projeto DAO.
- Vínculo com a Lista de Verificação:
  - Capacidade Avaliada: CT05 e CT06
  - Critérios Atendidos: 4.1 (Conexão bem-sucedida SQLite) e 5.1 (Estruturação e padrão DAO).

## **Aula 10 e Aula 16: Tratamento de Erros, Testes Unitários e Rastreabilidade**

- Conteúdo Teórico/Prático: Uso de abordagens para controle de falhas (`try/except/finally`). Construção de rotinas de qualidade utilizando o framework `unittest` para garantir estabilidade lógica antes da entrega da aplicação Web.
- Vínculo com a Lista de Verificação:
  - Capacidade Avaliada: CT06
  - Critérios Atendidos: 5.2 (Arquitetura defensiva e exceções) e 5.3 (Testes Unitários automatizados).